

Um Guia de Implementação Abrangente para o Projeto de Navegação Assistida por Visão Computacional

Este documento serve como um roteiro técnico detalhado para o projeto "Navegação Assistida por Visão Computacional (T3)". Ele foi projetado para desconstruir a tarefa em componentes gerenciáveis, fornecer uma estratégia de desenvolvimento clara, detalhar as bibliotecas e algoritmos essenciais e orientar o processo de avaliação e elaboração do relatório final. O objetivo é transformar a complexidade do projeto em um plano de ação estruturado, permitindo uma implementação focada e bem-sucedida.

Seção 1: Estratégia Fundamental e Configuração Inicial

Para abordar um projeto de alta complexidade como este, é crucial estabelecer uma estratégia de desenvolvimento sólida desde o início. Esta seção delineia uma abordagem de prototipagem iterativa, resolve o desafio inicial de aquisição de dados e apresenta os componentes de software centrais que formarão a base do sistema.

1.1 A Estratégia de Prototipagem Iterativa: Do Simples ao Robusto

A abordagem mais eficaz para evitar a sobrecarga e garantir progresso contínuo é seguir a recomendação prática do documento do projeto: "comece com um protótipo simples... e itere".¹ Esta estratégia foca na construção de um sistema funcional de ponta a ponta o mais rápido possível, que servirá como base para melhorias incrementais.

O primeiro objetivo é desenvolver um Produto Mínimo Viável (MVP) que integre os componentes essenciais, mesmo que em suas formas mais básicas. Este MVP deve consistir em:

- Um detector de objetos pré-treinado e leve (por exemplo, YOLOv8n).
- Um modelo de profundidade monocular pré-treinado e rápido (por exemplo, MiDaS Small).
- Um mecanismo de conversão de texto em fala (Text-to-Speech, TTS) simples e offline (por exemplo, pyttsx3).
- Um script principal que processa um arquivo de vídeo pré-gravado, aplicando a detecção, estimando a distância e gerando uma saída de áudio.

Uma vez que este protótipo inicial esteja operacional, o desenvolvimento prosseguirá por meio de iterações, adicionando gradualmente os recursos mais avançados recomendados no escopo do projeto.¹ O caminho de iteração pode incluir:

1. **Adição de Rastreamento de Objetos:** Para garantir a estabilidade da narração e a persistência dos objetos entre os frames.
2. **Filtragem de Distância:** Para suavizar as estimativas de distância ruidosas.
3. **Refinamento da Lógica de Risco:** Para criar um sistema de alerta mais sofisticado e contextual.
4. **Otimização de Desempenho:** Para atender aos requisitos de tempo real.

Essa abordagem garante que haja um sistema demonstrável em todas as fases do desenvolvimento, facilitando os testes e a validação contínua.

1.2 Aquisição de Datasets para Desenvolvimento Inicial

Um obstáculo comum no início de projetos de visão computacional é a falta de dados específicos. Antes que a montagem da câmera peitoral esteja finalizada, é possível e altamente recomendável iniciar o desenvolvimento utilizando datasets públicos.

A análise de datasets de pedestres disponíveis revela que muitos, como o Caltech Pedestrian Dataset ou o CityPersons, são capturados da perspectiva de um veículo, o que não reflete adequadamente a visão egocêntrica (primeira pessoa) necessária para este projeto.² Da mesma forma, datasets de vigilância com múltiplas câmeras, embora úteis para detecção, não capturam a dinâmica de movimento de um usuário caminhando.³

A solução reside em datasets egocêntricos. O **Ego4D** é um recurso de grande escala que contém milhares de horas de vídeo de atividades da vida diária capturadas a partir de uma perspectiva de primeira pessoa.⁴ Seu subconjunto, **EgoTracks**, é particularmente relevante, pois fornece anotações de rastreamento de objetos a longo prazo, o que é ideal para testar o módulo de percepção e rastreamento.⁵ Outro recurso valioso é o dataset **LookOut**, que se concentra especificamente em vídeo egocêntrico para navegação sem colisões e inclui dados

de profundidade, alinhando-se perfeitamente com os objetivos do projeto.⁷

A ação recomendada é baixar clipes de vídeo relevantes desses datasets. Eles fornecem cenários realistas de movimento em primeira pessoa, oclusões e uma variedade de ambientes internos e externos, permitindo o desenvolvimento e teste de todo o pipeline de software antes da coleta de dados personalizados.

1.3 Seleção de Componentes e Bibliotecas Essenciais

A escolha correta das ferramentas de software é fundamental. O escopo do projeto permite e incentiva o uso de bibliotecas existentes, desde que suas escolhas sejam justificadas.¹ A tabela a seguir resume os módulos principais do sistema, suas finalidades e as bibliotecas recomendadas para a prototipagem inicial e para versões mais avançadas.

É fundamental entender que a escolha de um componente impacta diretamente os outros. Por exemplo, um modelo de detecção de objetos computacionalmente pesado aumentará a latência de cada ciclo de processamento, reduzindo a taxa de quadros por segundo (FPS) geral do sistema. Isso, por sua vez, diminui a frequência com que a lógica de risco de colisão pode ser atualizada, tornando o sistema menos responsivo a mudanças rápidas no ambiente. Portanto, as escolhas de modelo não são apenas sobre precisão, mas representam decisões arquitetônicas fundamentais sobre a capacidade de resposta do sistema. O "estudo ablativo" mencionado no documento do projeto não é apenas uma etapa final de avaliação, mas uma atividade de design contínua para encontrar o equilíbrio ideal entre precisão e latência para o hardware alvo.¹

Tabela 1: Matriz de Seleção de Componentes e Bibliotecas Essenciais

Módulo	Finalidade	Recomendação para Protótipo	Opção(ões) Avançada(s)	Bibliotecas Chave
Percepção	Detectar objetos no quadro de vídeo.	YOLOv8n	RT-DETR, YOLOv8 com rastreamento (ByteTrack/OC-SORT)	ultralytics, supervision
Estimativa de	Estimar a distância para	MiDaS Small	MiDaS Large (DPT_Large),	torch, timm, opencv-pytho

Profundidade	cada pixel.	(MiDaS_small)	LeReS, DPT	n
Feedback Auditivo	Converter alertas de texto em fala.	pyttsx3	Coqui TTS	pyttsx3, TTS
Suavização de Dados	Estabilizar leituras de distância ruidosas.	Média Móvel Simples	Filtro de Kalman	numpy, filterpy

Seção 2: O Módulo de Percepção: Detecção e Rastreamento de Objetos

O primeiro componente ativo do pipeline é a percepção visual, responsável por identificar objetos relevantes no ambiente. Esta seção detalha a escolha de um detector de objetos e a implementação do rastreamento para garantir uma experiência de usuário estável e confiável.

2.1 Escolhendo um Detector de Objetos em Tempo Real: YOLOv8 vs. RT-DETR

O documento do projeto sugere explicitamente a comparação entre detectores como YOLOv8n e RT-DETR-r18, representando um clássico trade-off entre velocidade e precisão.¹

- **YOLOv8:** A arquitetura YOLO (You Only Look Once) é renomada por sua alta velocidade de inferência e facilidade de uso. A biblioteca ultralytics simplifica enormemente o processo de treinamento e inferência, permitindo que um modelo pré-treinado seja carregado e utilizado com poucas linhas de código.⁸ Para o protótipo inicial, um modelo leve como o yolov8n.pt é a escolha ideal, pois oferece um excelente equilíbrio entre desempenho e requisitos computacionais.
- **RT-DETR:** Os modelos DETR (DEtection TRansformer) e sua variante em tempo real, RT-DETR, utilizam mecanismos de atenção para alcançar alta precisão, especialmente em cenas com múltiplos objetos e oclusões. No entanto, eles podem ser mais exigentes

computacionalmente.⁹

A recomendação estratégica é iniciar com YOLOv8n. Ele é rápido, bem documentado e mais do que suficiente para validar o pipeline completo. Posteriormente, como parte do "estudo ablativo" exigido, o RT-DETR pode ser integrado para uma comparação quantitativa do impacto na precisão e na latência do sistema.¹

2.2 Implementando Rastreamento de Objetos para Estabilidade Temporal

O uso de um detector de objetos quadro a quadro é insuficiente para uma aplicação de navegação assistida. Um detector pode falhar em identificar um objeto em um único quadro devido a desfoque de movimento ou oclusão parcial. Sem um mecanismo de rastreamento, isso levaria a uma experiência de usuário intermitente e confusa, com o sistema anunciando repetidamente a presença e ausência do mesmo objeto (por exemplo, "Pessoa à frente... Siga em frente... Pessoa à frente..."). Isso mina a confiança do usuário no sistema, tornando-o ineficaz como uma ferramenta de auxílio.

O rastreamento de objetos, fortemente recomendado no escopo do projeto¹, resolve esse problema atribuindo uma identidade única e persistente a cada objeto detectado. Isso transforma o sistema de um simples "sensor" que reage a cada quadro isoladamente para um "guia" que possui uma memória de curto prazo e entende a continuidade do ambiente. Com um ID persistente, a lógica de narração pode ser muito mais inteligente (por exemplo, "A pessoa continua à sua frente"), e o histórico de posições de um objeto (seu *tracklet*) pode ser usado para estimar sua velocidade, um dado crucial para a lógica de risco de colisão.¹

- **ByteTrack:** Este algoritmo de rastreamento é particularmente robusto porque associa detecções mesmo com baixa pontuação de confiança, o que o torna eficaz em cenários com oclusões.¹⁰ Sua integração com o YOLOv8 é facilitada por bibliotecas como a *supervision*, que abstrai grande parte da complexidade.⁸ O processo envolve obter as detecções do YOLO, passá-las para o objeto rastreador e receber de volta as mesmas detecções, agora enriquecidas com um *tracker_id*.
- **OC-SORT:** Como alternativa, o OC-SORT (Observation-Centric SORT) melhora os rastreadores tradicionais baseados em movimento, sendo mais resiliente a movimentos não lineares e oclusões prolongadas, características comuns em vídeos egocêntricos.¹²

A implementação prática deve focar no uso de bibliotecas de alto nível como *supervision* ou *boxmot*¹⁴, que fornecem integrações prontas para uso com detectores populares, acelerando o desenvolvimento.

Seção 3: O Módulo de Profundidade: Estimando a Distância a Partir de uma Única Visão

Após identificar o *que* está no ambiente, o próximo passo é determinar *onde* está. Esta seção aborda a estimativa de profundidade monocular, uma técnica que adiciona a terceira dimensão à percepção 2D do sistema, permitindo a estimativa de distância.

3.1 Introdução à Estimativa de Profundidade Monocular com MiDaS

A estimativa de profundidade monocular é a tarefa de prever um valor de distância para cada pixel em uma única imagem RGB. O resultado é um "mapa de profundidade", onde a intensidade de cada pixel (ou cor, em uma visualização) corresponde à distância da câmera.¹⁵

A família de modelos **MiDaS**, desenvolvida pela Intel, é o estado da arte nesta área.¹⁶ Esses modelos são pré-treinados em uma vasta gama de datasets e funcionam de forma "zero-shot", ou seja, produzem resultados de alta qualidade em imagens de novos ambientes sem a necessidade de retreinamento.¹⁷

O projeto exige desempenho em "tempo real prático"¹, tornando a escolha da variante do MiDaS uma decisão crítica. Os modelos disponíveis via PyTorch Hub oferecem diferentes trade-offs¹⁶:

- **DPT_Large**: Oferece a maior precisão, mas é o mais lento.
- **DPT_Hybrid**: Um equilíbrio entre precisão e velocidade.
- **MiDaS_small**: A variante mais rápida, com a menor precisão, mas ideal para aplicações em tempo real em laptops ou sistemas embarcados.

A demonstração prática em vídeo do trade-off entre os modelos grande e pequeno ilustra perfeitamente este ponto: o modelo grande exibe um atraso visível, mas gera um mapa de profundidade mais detalhado e com gradientes mais suaves, enquanto o modelo pequeno é fluido e responsivo.¹⁸

3.2 Implementação Prática do MiDaS

A implementação do MiDaS pode ser realizada seguindo um processo claro, baseado nos tutoriais disponíveis ¹⁵:

1. **Carregar o Modelo:** Utilizar `torch.hub.load("intel-isl/MiDaS", model_type)` para baixar o modelo desejado (por exemplo, "MiDaS_small").
2. **Carregar a Transformação:** Cada modelo requer uma transformação de entrada específica para redimensionamento e normalização. Esta é carregada via `torch.hub.load("intel-isl/MiDaS", "transforms")`.
3. **Pré-processar a Imagem:** Ler um quadro do vídeo com OpenCV, convertê-lo para o formato de cor RGB e aplicar a transformação carregada.
4. **Realizar a Inferência:** Passar o tensor da imagem pré-processada pelo modelo para obter o mapa de profundidade bruto.
5. **Pós-processar a Saída:** O modelo gera um mapa de profundidade *inversa relativa*. Este mapa deve ser redimensionado para as dimensões da imagem original. Para fins de depuração, ele pode ser convertido em um mapa de cores, onde cores quentes (como amarelo) representam objetos próximos e cores frias (como roxo) representam objetos distantes. ¹⁵

Um ponto crucial que requer atenção de engenharia é que a saída do MiDaS é uma profundidade *relativa* e *inversa*, não uma distância métrica absoluta em metros.¹⁷ No entanto, o requisito R4 do projeto exige uma "Estimativa de distância (em metros ou faixas de proximidade)".¹ Isso cria uma lacuna que deve ser preenchida por um passo de **calibração ou escala**. O documento do projeto antecipa essa necessidade, mencionando "Descrever calibração/escala".¹ Uma abordagem heurística para isso seria usar suposições sobre a cena, como a altura conhecida da câmera em relação ao solo, para derivar um fator de escala. Alternativamente, a detecção de um objeto de tamanho conhecido (por exemplo, uma pessoa) poderia ser usada para calibrar a escala da profundidade. Esta etapa de calibração é uma fonte significativa de erro potencial e deve ser claramente descrita e suas limitações discutidas no relatório final. A sugestão de usar faixas de proximidade ("próximo", "médio", "longe") é uma alternativa prática e robusta caso a calibração para uma métrica absoluta precisa se mostre muito desafiadora.¹

Seção 4: Lógica Central: Fusão de Percepção, Profundidade e Tempo

Esta é a seção onde os módulos individuais convergem para criar inteligência acionável. Aqui, a informação sobre *o que* é um objeto (detecção), *onde* ele está (profundidade) e como seu

estado evolui ao longo do tempo (rastreamento) é integrada.

4.1 Fusão de Bounding Boxes com Mapas de Profundidade

O objetivo é derivar uma única estimativa de distância para cada objeto detectado. O método, conforme sugerido no documento do projeto, é usar a informação de profundidade dentro da caixa delimitadora (bounding box) do objeto.¹ O processo é o seguinte:

1. Para cada objeto rastreado, obter as coordenadas de sua bounding box.
2. Usar essas coordenadas para extrair a região correspondente do mapa de profundidade, que está alinhado com a imagem original.
3. Dentro desta região, há um conjunto de valores de profundidade. Calcular a média simples é suscetível a valores atípicos, especialmente nas bordas do objeto onde a estimativa de profundidade pode ser imprecisa. Uma abordagem mais robusta é calcular a **mediana** dos valores de profundidade. A mediana é menos sensível a outliers e tende a fornecer uma estimativa mais estável da distância do objeto.
4. O valor da mediana resultante é a **estimativa de distância bruta** para aquele objeto naquele quadro específico.

4.2 Suavização das Estimativas de Distância com um Filtro de Kalman

A estimativa de distância bruta obtida no passo anterior será inerentemente ruidosa. Pequenas variações na pose da câmera ou do objeto, bem como imperfeições no modelo de profundidade, farão com que a distância estimada flutue erratically entre os quadros. Isso resultaria em uma narração instável e pouco confiável (por exemplo, "Pessoa a 2.5 metros... Pessoa a 2.1 metros... Pessoa a 2.6 metros").

A solução para este problema é a filtragem temporal, com o **Filtro de Kalman** sendo a ferramenta canônica para esta tarefa.¹ O Filtro de Kalman é um algoritmo recursivo que estima o estado verdadeiro de um sistema a partir de uma série de medições ruidosas. Ele funde as medições observadas com um modelo de como o sistema se comporta para produzir estimativas que são mais precisas do que as medições individuais.²⁰

A aplicação de um Filtro de Kalman vai além de uma simples suavização matemática. O filtro opera em um ciclo de duas fases: predição e atualização.²¹ A fase de predição utiliza um *modelo de transição de estado* (a matriz nos exemplos de implementação²¹), que representa um modelo físico de como se espera que o objeto se mova (por exemplo,). Isso significa que

o filtro mantém uma estimativa não apenas da posição (distância), mas também da velocidade do objeto.

Esta capacidade tem duas implicações poderosas:

1. **Robustez a Falhas do Sensor:** Se a estimativa de profundidade falhar por alguns quadros (por exemplo, devido a um desfoque de movimento intenso), o filtro pode continuar a *prever* a posição do objeto com base em sua última velocidade conhecida, mantendo a continuidade do rastreamento.
2. **Estimativa de Velocidade:** A velocidade relativa é um componente chave da "Regra de risco".¹ O Filtro de Kalman fornece essa estimativa de velocidade como parte de seu estado interno, permitindo a construção de uma lógica de prevenção de colisão que é preditiva, em vez de puramente reativa.

Para a implementação, bibliotecas como a `filterpy`²³ oferecem uma classe `KalmanFilter` fácil de usar, ou uma implementação simples pode ser criada com NumPy com base em exemplos disponíveis.²¹ Para cada objeto rastreado, uma instância separada do filtro deve ser mantida para suavizar sua trajetória de distância.

Seção 5: O Módulo de Interação: Experiência do Usuário e Feedback Auditivo

Esta seção final do pipeline de processamento foca na comunicação das informações relevantes ao usuário. A eficácia do sistema depende não apenas da precisão técnica, mas também da clareza, pontualidade e utilidade do feedback auditivo.

5.1 Escolhendo um Mecanismo de Texto-para-Fala (TTS): `pyttsx3` vs. Coqui TTS

O sistema deve fornecer feedback em português brasileiro, e o escopo do projeto recomenda que a solução seja "offline preferível" para garantir baixa latência e operação independente de conexão com a internet.¹

- **pyttsx3:** Esta biblioteca é a escolha ideal para o protótipo inicial. É multiplataforma, funciona totalmente offline e possui uma API extremamente simples.²⁴ A lógica básica para gerar fala se resume a `engine = pyttsx3.init()`, `engine.say("texto")` e

engine.runAndWait()).²⁶ Ela atende aos requisitos essenciais com sobrecarga mínima de desenvolvimento.

- **Coqui TTS:** Esta é uma biblioteca mais avançada, baseada em deep learning, que oferece vozes de qualidade significativamente superior e sonoridade mais natural.²⁷ Inclui recursos avançados como clonagem de voz.²⁹ No entanto, sua configuração é mais complexa e seus requisitos computacionais são maiores. É uma excelente opção para uma iteração avançada do projeto, visando melhorar a qualidade da experiência do usuário, mas pode ser desnecessária para a validação inicial do sistema.

A recomendação é começar com pyttsx3 para estabelecer rapidamente o loop de feedback auditivo. Uma vez que o sistema central esteja estável, a migração para o Coqui TTS pode ser considerada como um aprimoramento.

5.2 Projetando a Lógica de Colisão e Narração

O design da interação auditiva é um problema de gerenciamento de carga cognitiva. O objetivo não é descrever verbalmente tudo o que a câmera vê, mas sim fornecer a *informação mínima necessária* para garantir segurança e consciência situacional. O silêncio do sistema é tão importante quanto sua fala; a ausência de um aviso deve transmitir segurança.

- **Regra de Risco de Colisão:** O projeto requer uma "regra simples de risco".¹ Uma implementação prática pode seguir estes passos:
 1. **Definir Zonas de Risco:** Dividir o campo de visão horizontal em três zonas: esquerda, centro e direita, conforme recomendado.¹ A zona central representa o caminho de colisão mais direto.
 2. **Calcular um Escore de Risco:** Para cada objeto, o risco pode ser inversamente proporcional à sua distância suavizada. Objetos na zona central e abaixo de um limiar de distância crítico (por exemplo, 1.5 metros) recebem a maior prioridade.
 3. **Gerar Alertas Direcionais:** Com base na zona em que o objeto se encontra, gerar alertas mais específicos, como "Obstáculo próximo à esquerda. Desvie à direita."
- **Lógica de Narração e Limitação de Taxa (Rate-Limiting):** Para evitar o "spam de voz"¹, é essencial implementar uma lógica de narração inteligente:
 1. **Hierarquia e Priorização:** Alertas de colisão críticos (por exemplo, "Pare!") devem sempre ter prioridade máxima, interrompendo qualquer narração informativa em andamento. Este é um mecanismo de segurança fundamental ("Failsafe").¹
 2. **Limitação de Taxa:** Anúncios informativos (por exemplo, "Pessoa a 3 metros") devem ser espaçados por um período de espera (por exemplo, 3-5 segundos) para evitar repetição excessiva. Apenas o objeto mais relevante (com o maior escore de risco) deve ser narrado.
 3. **Supressão Inteligente:** Se o estado de um objeto não mudou significativamente (em

classe ou faixa de distância), a narração sobre ele deve ser suprimida para reduzir a redundância.

Seção 6: Integração do Sistema, Avaliação de Desempenho e Relatório

A fase final do projeto consiste em unir todos os módulos em uma aplicação coesa, medir rigorosamente seu desempenho em relação aos critérios definidos e documentar os resultados em um relatório técnico de alta qualidade.

6.1 Integrando o Pipeline

A estrutura do loop principal da aplicação deve seguir uma sequência lógica de processamento para cada quadro de vídeo capturado:

Python

```
# Inicialização dos módulos
detector = initialize_detector()
tracker = initialize_tracker()
depth_model = initialize_depth_model()
tts_engine = initialize_tts_engine()
object_filters = {} # Dicionário para manter filtros de Kalman por ID de objeto

# Loop principal
while video_is_running:
    frame = capture_frame()

    # 1. Percepção
    detections = detector.predict(frame)
    tracked_objects = tracker.update(detections)

    # 2. Profundidade
```

```

depth_map = depth_model.predict(frame)

# 3. Lógica de Fusão e Suavização
for obj in tracked_objects:
    raw_dist = calculate_distance(obj.bbox, depth_map)

    # Obter ou criar filtro de Kalman para o objeto
    kalman_filter = object_filters.get(obj.id)
    if kalman_filter is None:
        kalman_filter = initialize_kalman_filter()
        object_filters[obj.id] = kalman_filter

    obj.smooth_dist = kalman_filter.update(raw_dist)
    obj.risk_score = calculate_risk(obj)

# 4. Lógica de Narração e UX
narration_logic.decide_and_speak(tracked_objects)

# (Opcional) Exibir telemetria no quadro
display_results(frame)

```

6.2 Métricas de Desempenho e Protocolo de Avaliação

A avaliação experimental deve ser sistemática e baseada nas métricas exigidas pelo projeto.¹ A tabela a seguir fornece um protocolo claro para medir cada uma delas.

Tabela 2: Métricas de Avaliação e Protocolo de Medição

Categoria da Métrica	Métrica Específica	Protocolo de Medição
Desempenho em Tempo Real	FPS (Quadros por Segundo)	No loop principal, registrar o tempo de início e fim do processamento de um quadro. . Calcular a média sobre um vídeo inteiro.
	Latência de ponta a ponta	Registrar o timestamp na captura do quadro ().

		Registrar o timestamp quando a chamada do TTS (por exemplo, runAndWait()) é iniciada (). Latência = .
Precisão do Alerta	TP/FP/FN	Criar um vídeo de teste curto com eventos de colisão conhecidos. Anotar manualmente os quadros onde um alerta <i>deveria</i> ser acionado (ground truth). Executar o sistema e comparar seus alertas com o ground truth para contar Verdadeiros Positivos, Falsos Positivos e Falsos Negativos.
Precisão da Distância	Erro de distância (RMSE/MAE)	Em um ambiente controlado, posicionar um objeto a distâncias conhecidas (por exemplo, 1m, 2m, 3m). Registrar a estimativa de distância suavizada do sistema. Calcular o Erro Quadrático Médio Raiz (RMSE) ou Erro Absoluto Médio (MAE) em relação ao ground truth.
Estabilidade da Narração	Estabilidade da classe (Taxa de Troca)	Para um único objeto rastreado ao longo de N quadros, contar quantas vezes seu rótulo de classe anunciado muda. Taxa de Troca = (Número de mudanças) / N. Uma taxa mais baixa é melhor.

6.3 Estruturando o Relatório Técnico Final

O relatório final deve seguir a estrutura sugerida no documento do projeto para garantir que todos os aspectos sejam abordados de forma clara e completa ¹:

1. **Introdução & Requisitos:** Apresentar o problema e listar os requisitos R1-R4.
2. **Arquitetura do Sistema:** Incluir um diagrama de blocos claro que mostre a interação entre os módulos de Percepção, Profundidade, Lógica e TTS.
3. **Percepção:** Detalhar a escolha do detector e do rastreador, a justificativa baseada em desempenho, as classes-alvo e os parâmetros utilizados.
4. **Profundidade/Distância:** Descrever o modelo MiDaS utilizado, o método de calibração/escala para obter distâncias métricas ou em faixas, e a implementação do Filtro de Kalman para suavização.
5. **Lógica de Risco & Narração:** Explicar a regra de risco, a hierarquia de narração e as estratégias de limitação de taxa para garantir uma boa UX.
6. **Experimentos & Métricas:** Apresentar os resultados coletados (conforme a Tabela 2) em tabelas e gráficos bem formatados. Incluir a análise do estudo ablativo (por exemplo, comparação de desempenho entre YOLOv8 e RT-DETR).
7. **Limitações & Próximos Passos:** Discutir de forma honesta os desafios encontrados (por exemplo, desempenho em baixa luminosidade, imprecisão da escala métrica) e sugerir trabalhos futuros.
8. **Reprodutibilidade:** Fornecer instruções claras, um script de execução (por exemplo, run.sh) e um arquivo de requisitos (por exemplo, requirements.txt) para permitir que outros executem o projeto.

Finalmente, é importante reconhecer que a avaliação do projeto é dupla. Os critérios de avaliação dão peso significativo não apenas à "Qualidade técnica" (30%), mas também ao "Desempenho prático... demo" (20%) e à "Clareza do relatório" (20%).¹ Isso significa que a demonstração em vídeo é um teste crucial da eficácia da interação humano-computador. Um sistema pode ter métricas quantitativas excelentes, mas se a demonstração revelar uma experiência de usuário confusa ou irritante, o projeto não terá atingido seu objetivo principal. Portanto, a fase final de desenvolvimento deve se concentrar tanto na otimização das métricas numéricas quanto no ajuste fino dos parâmetros que governam a experiência do usuário, como os limiares de alerta e os tempos de espera da narração, para criar um sistema que seja genuinamente útil.

Works cited

1. T3.pdf
2. Datasets for pedestrian detection: our top 10 - Innovatiana, accessed October 13,

- 2025, <https://www.innovatiana.com/en/post/dataset-for-pedestrian-detection>
3. Multi-camera pedestrians video – CVLAB – EPFL, accessed October 13, 2025, <https://www.epfl.ch/labs/cvlab/data/data-pom-index-php/>
 4. Ego4D: Around the World in 3,000 Hours of Egocentric Video – Meta AI, accessed October 13, 2025, <https://ai.meta.com/research/publications/ego4d-uns scripted-first-person-video-f rom-around-the-world-and-a-benchmark-suite-for-egocentric-perception/>
 5. Egocentric 4D Perception (EGO4D), accessed October 13, 2025, <https://ego4d-data.org/>
 6. EgoTracks – Ego4D, accessed October 13, 2025, <https://ego4d-data.org/docs/data/egotracks/>
 7. LookOut: Real-World Humanoid Egocentric Navigation – arXiv, accessed October 13, 2025, <https://arxiv.org/html/2508.14466v1>
 8. YOLOv8 Object Tracking: A How-To Guide – Roboflow, accessed October 13, 2025, <https://roboflow.com/how-to-track/yolov8>
 9. How to Train RT-DETR Object Detection Transformer on Custom Dataset – YouTube, accessed October 13, 2025, <https://www.youtube.com/watch?v=sBARif4DnL4>
 10. An Introduction to BYTETrack: Multi-Object Tracking by Associating Every Detection Box, accessed October 13, 2025, <https://datature.io/blog/introduction-to-bytetrack-multi-object-tracking-by-asso ciating-every-detection-box>
 11. how-to-track-and-count-vehicles-with-yolov8.ipynb – Colab – Google, accessed October 13, 2025, <https://colab.research.google.com/github/roboflow-ai/notebooks/blob/main/note books/how-to-track-and-count-vehicles-with-yolov8.ipynb>
 12. OC-SORT Tutorial: Critical Insights You Can't Miss! – Labellerr, accessed October 13, 2025, <https://www.labellerr.com/blog/oc-sort/>
 13. noahcao/OC_SORT: [CVPR2023] The official repo for OC-SORT: Observation-Centric SORT on video Multi-Object Tracking. OC-SORT is simple, online and robust to occlusion/non-linear motion. – GitHub, accessed October 13, 2025, https://github.com/noahcao/OC_SORT
 14. Real-Time Object Tracking with OC-SORT – YouTube, accessed October 13, 2025, <https://www.youtube.com/watch?v=bORBNxLLTC0>
 15. A Code Implementation of Monocular Depth Estimation Using Intel MiDaS Open Source Model on Google Colab with PyTorch and OpenCV – MarkTechPost, accessed October 13, 2025, <https://www.marktechpost.com/2025/03/27/a-code-implementation-of-monocul ar-depth-estimation-using-intel-midas-open-source-model-on-google-colab-wi th-pytorch-and-opencv/>
 16. isl-org/MiDaS: Code for robust monocular depth estimation described in "Ranftl et. al., Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer, TPAMI 2022" – GitHub, accessed October 13, 2025, <https://github.com/isl-org/MiDaS>
 17. MiDaS – PyTorch, accessed October 13, 2025,

- https://pytorch.org/hub/intelisl_midas_v2/
18. Depth Estimation with Single Camera using MiDaS - YouTube, accessed October 13, 2025, https://www.youtube.com/watch?v=UUzpepJ_mLk
 19. MiDaS Depth Estimation | Intel Technology - YouTube, accessed October 13, 2025, <https://www.youtube.com/watch?v=UjaeNNFf9sE>
 20. Kalman filter - Wikipedia, accessed October 13, 2025, https://en.wikipedia.org/wiki/Kalman_filter
 21. Kalman Filter in Python - GeeksforGeeks, accessed October 13, 2025, <https://www.geeksforgeeks.org/python/kalman-filter-in-python/>
 22. Kalman Filter Python: Tutorial and Strategies - QuantInsti Blog, accessed October 13, 2025, <https://blog.quantinsti.com/kalman-filter/>
 23. KalmanFilter — FilterPy 1.4.4 documentation, accessed October 13, 2025, <https://filterpy.readthedocs.io/en/latest/kalman/KalmanFilter.html>
 24. PYTTX3 | PDF - Scribd, accessed October 13, 2025, <https://www.scribd.com/document/697010400/PYTTX3>
 25. pyttx3 - PyPI, accessed October 13, 2025, <https://pypi.org/project/pyttx3/>
 26. Text to Speech by using pyttx3 - Python - GeeksforGeeks, accessed October 13, 2025, <https://www.geeksforgeeks.org/python/python-text-to-speech-by-using-pyttx3/>
 27. Coqui TTS: Deep Dive Into an Open-Source Text-to-Speech Framework | by Sudeshnm, accessed October 13, 2025, <https://medium.com/@sudeshnm/coqui-tts-deep-dive-into-an-open-source-text-to-speech-framework-129c76a66580>
 28. coqui-tts 0.27.0 documentation, accessed October 13, 2025, <https://coqui-tts.readthedocs.io/>
 29. Synthesizing speech - coqui-tts 0.27.0 documentation, accessed October 13, 2025, <https://coqui-tts.readthedocs.io/en/latest/inference.html>